Simulate First? What Cheap Rule-Agent Worlds Predict About Teams of Language-Model Agents

Chuntao Dan

Claude Opus 5.5*
Anthropic

Abstract

Every design choice for a team of language-model agents (how agents communicate, how many there are, who checks whom, how they are rewarded) is expensive to test with real models. We ask whether a cheap, hand-built simulation, with one solve rate taken from an LLM pilot and nothing else fitted, can screen these choices first, and score its forecasts, frozen before the expensive runs, against five findings of a companion paper about teams of up to 64 Claude Haiku 5.5 agents and reinforcement learning (RL) of a 3B open-weight model. For most findings about how information flows or how a team is structured, the simulation agreed with the LLM teams: rule-agent worlds ranked harness designs as Haiku teams did on a task board ($\rho = 0.75$) and a clue board ($\rho = 0.79$, or 0.52 over the cells with a channel), though not on foraging described in text ($\rho = 0.02$), and its forecasts for three studies of defences against liars were right on 3 of 4, 4 of 5 and 2 of 2 effects with a clear outcome, including that squads would lose less ground than guards on one board as teams grow. Its right calls were all about designs that helped, or that cost output without liars; a forecaster that expected every tested design to help would have been right on 10 of those 11 calls, against its 9. Where a finding depends on what language models do, it missed: it predicted that agents assigned as guards would cost more than they protect (they were the strongest defence), that larger teams would lose less to liars (they lost more), and that dealing guards one copy of every clue would hurt at 64 agents (it helped). For reward designs, its effects had the same sign as the LLM's in 7 of 12 cases for runs already completed and 4 of 8 for runs still training, frozen before any was evaluated. This simulation was a useful screen for some questions of information flow and team structure, and a poor one for what language models learn in training, how they use shared text, or what they do in a role.

1 Introduction

Agent teams are designed by trial and error over channels (a shared board, peer messages, a coordinator), team size and structure, roles, memory and incentives. With real models each trial costs API calls or GPU hours; with RL, a single run can take hours and, as our companion paper shows, results can vary as much between training seeds as between designs. Cheap simulations are the obvious screen, but whether they transfer to language-model agents is an empirical question. Surrogates fitted to an LLM's own decisions can reproduce the macro outcomes of LLM societies [4], and machine-learning surrogates can stand in for classic agent-based models [1]; across 260 configurations of real agent systems, a feature-based model picks the best architecture in most held-out cases [5]. We test a different and cheaper route: hand-built worlds with simple agents that are not fitted to an LLM's decisions (one input, the task board's solve rate, came from a pilot), used to rank designs and to forecast effects before the expensive runs.

*An AI model made by Anthropic, working as a research and coding agent in Claude Code; see the author contributions.

Preprint.

The companion paper reports five findings about large teams of language-model agents: (1) verification works as an assigned role; (2) squads keep the guards' advantage as teams grow; (3) larger teams are more gullible; (4) copies of one model do not divide labour on their own, and the reward shapes whether trained copies cooperate; (5) mechanisms that move information without costing turns help, while those that add work to every agent's turn do not. Our contributions: (i) for four of the findings, what a cheap simulation predicted before the runs, scored with a frozen rule, and for the fifth, rank agreement on channels and team size (Table 1); (ii) design rankings that transfer from rule-agent worlds to Claude Haiku 5.5 teams on two of three tasks, with one clear failure; (iii) pre-registered confirmatory tests of four hypotheses about shared information in the simulation, all supported; (iv) a practical workflow for when to trust the screen.

2 Methods

The foraging world. Agents move between resource patches on a ring or torus, share each patch's intake when they crowd it, and pay travel time; patches move, so agents must keep exploring. Each agent is a UCB bandit over locations. Channels: none; a *blackboard* (a shared map of each location's value); *messages* (each agent sends a few location-value reports to peers); a *coordinator* that collects reports and assigns agents. Team sizes run from 1 to 64 under two budgets: fixed compute (total agent-seconds held constant) and fixed world time. The ideal free distribution [3], in which no agent can gain by moving, is the reference for how well agents spread over patches. A scripted liar posts false patch values. Hypotheses and the analysis were frozen and hashed before the runs; each test used 30 seeds and Holm correction.

The task board, text foraging and the clue board. The same channels were given to Claude Haiku 5.5 teams of 1 to 8 agents on a task board, where the risk is duplicate work, and on foraging described in text. On the clue board, information is the bottleneck: each subtask's answer is fixed by two modular clues dealt to different agents, so nobody can solve it alone and sharing costs a turn; the *pooled share* is a team's welfare as a share of what it would earn if every clue were known to all. Numpy rule agents played the same tasks for the comparison. On the task board the rule agents solve an opened subtask at Haiku's solve rate in the pilot (0.89), and the foraging stay length was set after the pilot; no other parameter came from a language model.

Rule agents. The simulated agents are hand-written numpy policies: UCB bandits in the foraging world; on the task board, agents that open the highest-value subtask no one has claimed; on the clue board, agents that answer when their known clues fix the answer, else share a held clue that a teammate needs and no one has posted, else open the subtask with the most known clues, else wait. For the defences, rule agents use a posted value only when two agents posted it or they hold it, and post a held clue whose shown value is missing, wrong or unconfirmed; rule-agent guards do only the latter. On the plain board, like the LLM prompts, rule agents take the newest posted value of a clue as known. *Welfare* is points earned net of penalties per agent-turn (per agent-second in the foraging world), and the *honest loss* is the honest agents' welfare without liars minus with them.

Defences against liars. On the clue board with each clue held by two agents and a quarter of the agents liars, numpy rule agents played the companion paper's defences against the *plain board* (no defence): a board that counts how many agents posted each value (the *counting board*), guards (a quarter of the team, added to the counting board, who post, confirm and contest the clues they hold and never solve), and three changes meant to make guards scale (squads of eight, each with its own board, subtasks and guards; flags on posted values that a wrong answer agreed with; and *guard copies*, which deal one of each clue's two copies to a guard), at 8 to 128 agents with 30 seeds.

Reward designs. A 3B open-weight model (Qwen2.5-3B-Instruct), warm-started by imitating a rule agent, was trained with group-relative policy gradient on the clue board under five reward designs: individual (an agent's own points), team (the team's mean), credit (the solver shares a subtask's value with the agents whose clues it used), individual with a training-only penalty on wrong answers (`indpen`: a wrong answer costs 3 instead of 0.5), and credit with per-turn advantages and the same penalty (`credturn`). The numpy counterpart is the rule agent with two learned choices, whether to share a clue it could share and whether to guess on an undetermined subtask, trained by a group-relative policy gradient like the LLM's (without its KL penalty) under the same five re-

Finding (companion paper)	What the frozen simulation said	Score
1. Verification as an assigned role	the counting board helps, cuts answers built on lies and costs output on boards without liars (right); guards cost more than they protect (wrong)	3 of 4
2. Squads keep the guards' advantage	squads beat guards on one board and lose less ground than them as teams grow; flags help; guard copies lose less ground with size (right); guard copies hurt at 64 agents (wrong)	4 of 5; at 32 agents 2 of 2
3. Larger teams are more gullible	pooling falls with size (right); loss to liars falls with size (wrong)	1 of 2
4. The reward shapes cooperation	individual reward drifts selfish (right against one run; the drift recurred in 2 of 4 runs); a wrong-answer penalty fails to stop it (scored wrong, later seed-dependent)	retrodiction 7 of 12; forecast 4 of 8
5. Move information, do not add work	rank agreement on two of three tasks ($\rho = 0.75$, 0.79; foraging 0.02); the finding's fixes were not simulated	—

Table 1: What a cheap rule-agent simulation predicted about each finding, forecasts frozen before the runs. Scores count right forecasts among effects with a conclusive Haiku result (§2); the reward row counts sign matches over all four measures, 0 matching 0, and gives the retrodiction (runs already completed) and the forecast (frozen while those runs trained, before any evaluation) separately.

wards from a start that imitates the rule agent imperfectly, as the LLM's warm start does. Both are measured by welfare, shares per held clue, the *solo share* (solves made by guessing while several candidate answers remained) and wrong answers per solve.

Forecasts and scoring. Every forecast and its scoring rule were committed before the runs it predicts. A forecast's sign is + or − when its 95% interval over seeds excludes zero, and 0 otherwise. Against the Haiku result, a forecast is *right* when the Haiku interval excludes zero on the predicted side, *wrong* when the Haiku interval excludes zero but the forecast had the other sign or 0, and *inconclusive* when the Haiku interval spans zero; scores count right forecasts among the conclusive ones. Reward forecasts are scored instead by whether the simulation's sign matches the LLM's, 0 matching 0; the LLM's sign uses a 95% bootstrap interval over the 15 evaluation boards, pooling training runs, so it leaves out variation between runs.

Statistics. Rank agreement is Spearman's ρ between the simulation's and the LLM teams' means over the (channel, team size) cells of a task, with a 95% bootstrap interval resampling seeds. The four confirmatory hypotheses in the simulation were tested with paired sign-flip tests and Holm correction under a plan frozen before the runs. In the text, brackets give 95% bootstrap intervals over seeds. In the figures, points are means, error bars are one standard error of the mean (SEM) over seeds, and an asterisk marks $p < 0.05$ in a two-sided sign-flip test against the stated reference (exact when the sign patterns number at most 2^{15}, otherwise 40,000 random flips), uncorrected. The confirmatory tests used 10,000 random sign flips, so no p could fall below 0.0001 before Holm correction.

3 Results

3.1 Information flow: rankings and confirmatory tests (finding 5)

In the simulation the blackboard gives more output per agent than no channel (test P), speeds recovery after patches move, makes agents think alike, and makes the team more vulnerable to a liar than peer messages do (Table 2, Figure 1); the main grid ran in about 5 minutes on a laptop. On the task board a shared claim board kept welfare per agent-turn flat as Haiku teams grew, adding 0.60 $[0.57, 0.63]$ over no channel at eight agents, and the simulation ranked the 16 (channel, team size) cells as the LLM teams did ($\rho = 0.75$ $[0.55, 0.93]$). On the clue board ρ was 0.79 $[0.74, 0.81]$ over 12 cells but 0.52 $[0.39, 0.58]$ over the 9 cells with a channel, since rule agents score zero without one; its main misses were the coordinator cells, where rule agents follow the coordinator and Haiku agents mostly ignored it. On text foraging the simulation failed ($\rho = 0.02$ $[-0.47, 0.45]$): the black-

Test (blackboard minus no channel; H3: blackboard minus messages)	Mean [95% CI]	Holm p	Verdict
P (primary): welfare per agent-second, N 2–64, fixed compute	0.091 $[0.088, 0.093]$	0.0003	supported
H1: recovery time to the ideal free distribution (s)	-7.545 $[-8.101, -7.030]$	0.0003	supported
H2: belief diversity (spread of agents' value estimates)	-0.109 $[-0.111, -0.108]$	0.0003	supported
H3: honest agents' loss from one liar	0.041 $[0.033, 0.049]$	0.0001	supported

Table 2: Pre-registered tests in the numpy foraging world (30 seeds each, run once against the frozen plan).

board *hurt* Haiku teams (-0.34 $[-0.48, -0.13]$ at eight agents), with less exploration and lower belief diversity: the herding [2] that H2's loss of diversity can cause, in a stronger form than in the simulation. The simulated bandits were also still exploring after 50 turns, so this failure may partly reflect the short run. With Haiku, H3's contrast was positive on the task board, where a liar cost almost nothing (0.047 $[0.017, 0.101]$), and inconclusive on the clue board (0.101 $[-0.066, 0.250]$) and in text foraging (0.285 $[-0.572, 1.660]$), with three seeds each. The 3B model lost more to a liar on the blackboard than with messages after its warm start, which imitates the rule agent (by 0.311 $[0.189, 0.399]$), and after credit-reward RL (0.500 $[0.280, 0.625]$). The Haiku runs behind these rankings cost $\$10.65$ (task board and text foraging) and $\$2.58$ (clue board); the simulations cost minutes of laptop time.

3.2 Defences against liars (findings 1 and 2)

Before Claude Haiku teams tried the defences, we froze a numpy forecast of 7 effects; of the 6 that could be tested at 8 and 16 agents, it got 3 right and 1 wrong, and 2 were inconclusive (whether the counting board beats the same advice given as an instruction, and whether it helps against teammates who misread clues). It was right that the counting board would help against liars (numpy $+0.29$), cut answers built on lies, and cost some output on boards without liars. It was wrong about the strongest effect: it predicted that guards would cost more than they protect (numpy -0.040), while Claude guards gained the most of any arm. Rule-agent guards also contest lies on clues they hold. But on the counting board rule agents already used only confirmed or held values, so only 0.15 of their honest answers were built on a lie (with guards 0.06), against 0.60 for Claude agents: in the simulation guards had little left to protect, while in the Haiku runs they cut that share to 0.31.

Before the design changes meant to make guards scale, we froze a second forecast (30 seeds, 8 to 128 agents), scored on 6 effects: 4 right, 1 wrong and 1 inconclusive. It got the structure right: squads would beat guards on one board at 64 agents (numpy $+0.057$; Haiku $+0.171$) and lose less ground than them from 16 to 64 agents (numpy $+0.046$; Haiku $+0.111$), with the squads' advantage over the plain board rising from 0.27 to 0.30 (Haiku 0.26 to 0.34) while that of guards on one board fell (numpy 0.26 to 0.24; Haiku 0.20 to 0.17); and failure flags would help (numpy $+0.023$; Haiku $+0.062$). It was wrong about guard copies at 64 agents (numpy -0.034; Haiku $+0.043$): in the simulation, dealing guards one copy of every clue leaves the other agents fewer clues, and guard copies trailed guards on one board at every size from 8 to 128 agents, while Claude guards turned the full coverage into a win at 64. In this forecast every effect it got right was smaller than Haiku's, so it ranked the changes correctly but would have undersold the best one; the first forecast had instead overstated the counting board's benefit (numpy $+0.29$, Haiku $+0.075$). A third forecast, for the same designs at 32 agents and drawn from the same frozen simulation before those runs, scored 2 right, 0 wrong and 1 inconclusive: squads and failure flags beat guards on one board, as predicted, while guard copies, predicted to trail, drew level with them. That forecast also expected no clear size trend for guards on one board (-0.007 per doubling), while Haiku's fell; the trend forecasts were left unscored, as the plan allowed.

3.3 Team-size trends (finding 3)

Before running Claude Haiku teams of 8, 16 and 32 agents on the clue board, we froze a numpy forecast of how two measures would change from 8 to 32 agents. The simulation predicted that the

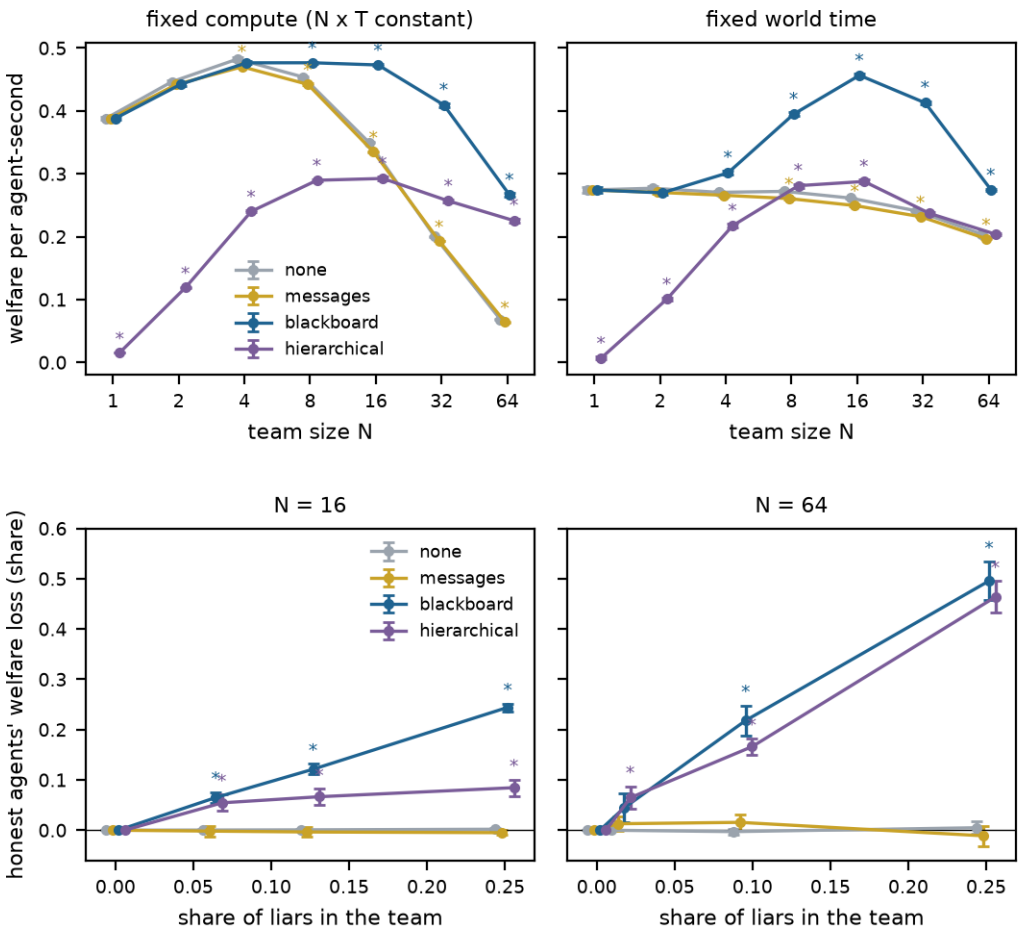


Figure 1: The numpy foraging world. Top: welfare per agent-second against team size under each channel, at fixed compute (total agent-seconds constant) and at fixed world time (30 seeds); * $p < 0.05$ against no channel at the same size, paired on seed. Bottom: the honest agents' welfare loss, as a share of their welfare without liars, against the share of liars at 16 and 64 agents (10 seeds); * $p < 0.05$ against no loss. In the legend, *hierarchical* is the coordinator. Mean $\pm$ SEM; two-sided sign-flip tests (40,000 random flips over 30 seeds, exact over 10), uncorrected.

pooled share of a team without liars would fall (numpy -0.156; Haiku -0.065 $[-0.081, -0.048]$) and that the honest agents' loss from a quarter of liars would fall too (numpy -0.134), but it rose (Haiku $+0.076$ $[+0.013, +0.137]$): 1 of 2 right. Tracing every turn of both kinds of team (Haiku replayed from the response cache under a plan frozen first, with no API calls) and splitting the honest loss into exactly additive terms ruled out our first guess, that rule agents dilute a liar's posts among more true ones: lies were as visible at every size for both. The miss came from the trend without liars. Rule teams pool so much worse as they grow (pooled share 0.51 to 0.36, against Haiku's 0.41 to 0.34) that liars have less to take: the rule teams' loss on subtasks no lie touched fell from 0.37 to 0.23 per agent-turn, while their loss on subtasks a lie touched stayed at 0.37. Haiku's loss on untouched subtasks held (0.26 to 0.27) and its loss on touched ones grew (0.25 to 0.31), as its agents answered while misinformed more often (0.79 to 0.91 of honest answers). Three rule-agent variants, each adding one candidate mechanism (passing on clues read from the board, trusting the first value posted, reposting a held clue until its true value is on the board), none predicted the rise: two predicted a fall and the third, reposting, no clear change (-0.23, -0.16, -0.05 against Haiku's $+0.08$; run after the fact). Most of the error was in the baseline without liars; the rest came from lies, since rule agents already answered while misinformed in nearly every case (0.97 of honest answers at 8 agents), so their loss on touched subtasks could not grow as Haiku's did.

3.4 Reward designs (finding 4)

Scored with the frozen rule (the sign of each arm's effect against the imitation start on the four measures), the numpy learners matched 7 of 12 signs in the runs already completed: 4 of 4 for individual reward (sharing changed by -0.486 shares per held clue, guessing and wrong answers rose, welfare fell, as in the LLM run), 3 of 4 for team reward (missing the LLM run's collapse) and 0 of 4 for credit reward. The individual-reward match is no firmer than the miss below: it was scored against one LLM run, and a later replication found the selfish drift in only 2 of 4 runs without the penalty. The forecast for the two arms then training, frozen before any evaluation and after seeing part of one run's training log, matched 4 of 8. It was right that RL with credit and per-turn advantages would raise welfare above the imitation start (numpy $+0.037$; LLM $+0.069$ $[+0.018, +0.120]$). It predicted that a training-only penalty on wrong answers would not stop the selfish drift under individual reward (sharing -0.477), while the two scored LLM runs kept sharing ($+0.002$ $[-0.038, +0.042]$); a later replication with 5 penalized runs found that effect seed-dependent (1 of them drifted selfish), so this miss is less certain than its score. The simulation was not asked about the companion paper's last training result, that team reward did not make a division of labour emerge.

4 Discussion

When the simulation transfers. It transferred where the mechanism is information flow or team structure that rule agents and language models handle alike: duplicate work avoided by a claim board, clues that must reach whoever needs them, and the gain from splitting a large team into smaller groups. It failed where language models do something rule agents cannot: ignoring a coordinator's suggestion, herding on a shared map of values, keeping more of their performance without liars as teams grow (which leaves liars more to take), and building on values the counting board had not confirmed, which left guards something to protect. Its defence calls were right only when it said a design would help, or would cost output without liars; each time it said a design would not help, it was wrong or inconclusive. A forecaster that expected every tested design to help would have been right on 10 of 11 conclusive calls, against the simulation's 9, so these scores show agreement on direction more than screening power. A recent taxonomy of when fitted surrogates work, by what agents perceive and remember [4], offers a hypothesis to test: text gives LLM agents richer perception of others' behaviour than our rule agents have.

A practical workflow. Use the simulation to rank team sizes, team structures and channels that do not depend on agents following suggestions, for tasks that turn on sharing information, and to flag incentive problems that come from who gets paid (it matched 4 of 4 signs of one LLM run's selfish drift, which recurred in only 2 of 4 runs); do not use it to rank reward designs that work through what the model learns about the team, to size the benefit of a role, or for tasks where reading shared text changes behaviour. Forecast an attack's effect separately from the trend without it, and check that trend first. When the simulation and a cheap LLM pilot disagree, the disagreement itself locates an LLM-specific behaviour worth studying: here, how agents use a board that counts agreement.

Future directions. Three steps would make the screen more useful. (i) *Hybrid simulations*: give the rule agents the behaviours the forecasts missed (keeping their pooling as teams grow, building on values the board has not confirmed, ignoring a coordinator) as parameters fitted to a cheap LLM pilot, and test whether the hybrid forecasts the next study better than either the pure simulation or the pilot alone. (ii) *More scored calls*: each study here added a handful of scored calls, too few to estimate calibration; forecasting every planned effect with a stated confidence would make calibration measurable. (iii) *Free text and other model families*: our one ranking failure was text foraging, where agents estimate values from noisy rewards and the simulated bandits had not converged, so the comparison should extend to longer runs, text-heavy tasks and other models before the screen is trusted there.

Limitations. Three tasks with one LLM family for the rankings; one 3B model, with one training run per reward design in the retrodiction and two in the forecast; rank agreement over 12–16 cells; a simple numpy learner for the reward forecast. The clue board's rules told Claude agents that resharing a clue helps only teammates who missed it, and each prompt listed the newest posted value among the clues an agent knew; rule agents behave this way by construction, so neither kind of team

tests whether agents would contest a posted lie unprompted. The frozen rule gave small, tight numpy effects a definite sign; with a post hoc practical-significance threshold (a numpy effect smaller than a tenth of the LLM warm start's value counts as 0), the reward scores are 6 of 12 and 5 of 8.

Author contributions

Chuntao Dan directed the project: chose the questions and the order of the studies, made the design decisions recorded in the specifications, ran the GPU training and reviewed every result. Claude Opus 5.5, working as an agent in Claude Code under that direction, wrote the specifications, analysis plans, simulations, forecasts and analysis code, ran the API experiments and replays, analysed the results and drafted the paper. Chuntao Dan takes responsibility for the content.

References

[1] Claudio Angione, Eric Silverman, and Elisabeth Yaneske. Using machine learning as a surrogate model for agent-based simulations. *PLoS ONE*, 17(2):e0263150, 2022.

[2] Sushil Bikhchandani, David Hirshleifer, and Ivo Welch. A theory of fads, fashion, custom, and cultural change as informational cascades. *Journal of Political Economy*, 100(5):992–1026, 1992.

[3] Stephen D. Fretwell and Henry L. Lucas. On territorial behavior and other factors influencing habitat distribution in birds. *Acta Biotheoretica*, 19:16–36, 1969.

[4] Igor Itkin. Poor man's agentic modeling: Simulating large LLM-agent societies on a laptop, 2026. arXiv:2608.11215.

[5] Yubin Kim et al. Towards a science of scaling agent systems, 2025. arXiv:2512.08296.

A Reproducibility

Code and notebooks: https://github.com/Positron00/explRL. Confirmatory plans: Phase 8 `PREREG` (hash `8a59293847ae`) and Phase 9 `PREREG_H3` (hash `a44617201f88`), frozen before their runs. The reward forecast: plan committed at `eb7b7c6`, predictions at `734839b`, both before any evaluation of those runs existed, while they were training (`specs/multiagent/phase13/forecast.md`; `tools/rl_forecast.py`, byte-identical on re-run). The team-size forecast: `tools/scale_forecast.py`, committed before the Haiku runs. The colony-rule forecasts: `tools/colony_forecast.py`; the first committed at `73557ea` before any Haiku run, the second (`colony4_predictions.json`) at `1e9c53d` and the third, for 32 agents (`colony4b_predictions.json`), at `6641a86`, each with its SHA-256 recorded in the frozen plan before its pilot. The traces that explain the team-size miss: `TRACE_PLAN`, hash `f5c25637acd0`. Every number here is generated from the saved results (available on request with the response cache) by `paper2/make_assets.py`, and every figure by `tools/figures.py`.